

Project 1 -- Concurrency

Project Administration and Policies

Due date: Sunday, October 18th, 2026, 11:59 PM

Late Days: Projects will continue to be graded for three days past the due date and will not be accepted after the third day. Project grades receive a 10 percentage point penalty for each day late.

Grading: The `solution` folder is what is pulled from your repository when running the grader. Your score is proportional to the number of tests you pass.

Submission: Compress your project into a single `.zip` file.

- The zip file should contain the following structure:

```
123090000 (your School ID)/  
├─ solution/  
└─ tests/
```

Contact: If you have any questions regarding the project, please feel free to email danyangchen@link.cuhk.edu.cn.

Project Overview

Imagine a Tech Summit 2026, hosted by the SDS at the CUHKSZ. Hundreds of researchers, engineers, and students gather for a week-long conference on Computer Science.

To keep everyone updated, the organizers maintain a shared digital schedule board that displays the list of ongoing talks, workshops, and lunch specials. This board, however, has recently suffered race conditions leading to corrupted data (e.g., talks starting before attendees are notified).

Ensuring safe concurrent access—without deadlocks—is your mission.

This project simulates a tech conference with two classic concurrency problems:

1. **Readers-Writers** (shared schedule board)
2. **Producer-Consumer** (snacks distribution with bounded buffer)

Good luck with your implementation! 🚀

Part 1: Implement Reader-Writer Lock (30 points)

YOU MUST IMPLEMENT a **Reader-Priority Reader-Writer Lock** that allows:

- **Multiple readers** to access shared data simultaneously
- **Only one writer** to access shared data at a time
- **Readers get priority** over writers (as long as no writer is active, readers may enter without being blocked by waiting writers)

The Scenario

In the conference simulation, the **schedule board** is shared between:

- **Readers (Attendees):** Hundreds of attendees need to check the latest schedule at any time. To ensure the smoothest attendee experience, Part 1 uses a **Reader-Priority** strategy: as long as no writer is currently active, all readers can access the board concurrently with zero delay, unblocked by queued waiting writers.
- **Writers (Organizers):** Few people updating the schedule (need exclusive access, must wait for all currently reading attendees to leave before writing).

Implementation Requirements (YOU MUST COMPLETE THESE)

1. Complete the `rwlock_t` Structure

⚠ REQUIRED

File: `solution/include/sync_utils.h`

YOU MUST ADD the necessary fields to this structure:

```
typedef struct {  
    /* TODO: add semaphores/mutexes and counters */  
    // You need to add the required fields here  
} rwlock_t;
```

Required fields to add:

- `pthread_mutex_t m` - Protects shared variables
- `sem_t wlock` - Writer lock semaphore (binary: 0 or 1)
- `int readers` - Count of active readers
- `int readers_waiting` - Count of waiting readers
- `int writers_waiting` - Count of waiting writers
- `boolean writer_active` - Flag indicating if a writer is active
- `pthread_cond_t ok_to_read` - Condition variable for waiting readers

2. Implement Reader-Writer Lock Functions

⚠ REQUIRED

File: `solution/src/readers_writers.c`

YOU MUST IMPLEMENT these four functions.

`rw_rlock()` - Reader Lock (YOU IMPLEMENT)

What you must do:

- Block readers only while a writer is active (reader-priority, not blocked by waiting writers)
- Increment reader count
- Use proper mutex locking/unlocking

`rw_runlock()` - Reader Unlock (YOU IMPLEMENT)

What you must do:

- Decrement reader count
- If this is the last reader AND writers are waiting, signal them
- Use proper mutex locking/unlocking

`rw_wlock()` - Writer Lock (YOU IMPLEMENT)

What you must do:

- Increment `writers_waiting`
- Wait in a loop while `writer_active` is true OR `readers > 0`
- When you get access, decrement `writers_waiting` and set `writer_active = true`
- Use proper mutex locking/unlocking and semaphore signaling

`rw_wunlock()` - Writer Unlock (YOU IMPLEMENT)

What you must do:

- Set `writer_active = false`
- If readers are waiting, wake all waiting readers first using the condition variable (reader-priority)
- Otherwise, if writers are waiting, signal one using `sem_post(&wlock)`
- Use proper mutex locking/unlocking

3. Initialize the Lock Structure

⚠ REQUIRED

File: `solution/src/sync_utils.c`

YOU MUST IMPLEMENT `rw_init()` and `rw_destroy()` to:

- Initialize all mutexes, condition variables, and semaphores
- Set initial counter values
- Clean up resources on destroy

Part 2: Bounded Buffer (Producer-Consumer) (40 points)

⚠ YOUR TASK

The Scenario

Conference snacks are managed using a bounded buffer for pickup and real-time inventory monitoring:

- **Producers (Cooks):** Prepare food trays and place them into the queue in First-In-First-Out (FIFO) order
- **Consumers (Attendees):** Take food trays from the queue, enjoy them, and **free the memory**
- **Floor Monitors (Supervisors):** Safely query the current number of trays on the buffet table in real time (via `bb_count`) to monitor stock and prevent shortages or waste
- **Bounded Buffer:** Fixed-size queue (capacity 8) holding food tray pointers

1. Complete the `bb_t` Structure

⚠ REQUIRED

File: `solution/include/sync_utils.h`

First, understand the food tray structure (already defined for you):

```
typedef struct {
    int tray_id;           // Unique tray identifier
    char *food_name;       // Name of food on tray (dynamically allocated)
    int prepared_by;       // Cook who prepared it
} food_tray_t;
```

YOU MUST ADD the necessary fields to the buffer structure:

```
typedef struct {
    /* TODO: add buffer array, semaphores, mutex, and indices */
    // You need to add the required fields here
} bb_t;
```

Required fields to add:

- `food_tray_t **buf` - Dynamic buffer array to store food tray **pointers**
- `int cap` - Capacity of the buffer
- `int head, tail` - Circular queue pointers (head for consumers, tail for producers)
- `int count` - Current number of food trays in the buffer
- `sem_t empty` - Semaphore counting empty slots (initialized to capacity)
- `sem_t full` - Semaphore counting full slots (initialized to 0)
- `pthread_mutex_t m` - Mutex to protect updates to `head / tail / count`

2. Implement Bounded Buffer Operations

⚠ REQUIRED

File: `solution/src/sync_utils.c`

YOU MUST IMPLEMENT these functions:

`bb_init()` - Initialize the buffer (YOU IMPLEMENT)

What you must do:

- Allocate memory for the buffer array using `calloc(capacity, sizeof(food_tray_t*))`
- Initialize `head`, `tail`, and `count` to 0
- Initialize `empty` semaphore to `capacity` (all slots empty initially)
- Initialize `full` semaphore to 0 (no slots full initially)
- Initialize the mutex `m`
- Return 0 on success, -1 on failure

`bb_destroy()` - Clean up resources (YOU IMPLEMENT)

What you must do:

- Destroy the mutex
- Destroy both semaphores (`empty` and `full`)
- Free the buffer array memory

bb_put () - Producer adds food tray (YOU IMPLEMENT)

Signature: `void bb_put(bb_t *q, food_tray_t *tray)`

What you must do:

1. Wait on `empty` semaphore (blocks if buffer is full)
2. Lock the mutex
3. Place tray pointer at `buf[tail]`
4. Update `tail = (tail + 1) % cap` (circular buffer) and increment `count`
5. Unlock the mutex
6. Post to `full` semaphore (signal one more item available)

bb_take () - Consumer removes food tray (YOU IMPLEMENT)

Signature: `food_tray_t* bb_take(bb_t *q)`

What you must do:

1. Wait on `full` semaphore (blocks if buffer is empty)
2. Lock the mutex
3. Retrieve tray pointer from `buf[head]`
4. Update `head = (head + 1) % cap` (circular buffer) and decrement `count`
5. Unlock the mutex
6. Post to `empty` semaphore (signal one more slot available)
7. Return the tray pointer

bb_count () - Query current buffer count (YOU IMPLEMENT)

Signature: `int bb_count(bb_t *q)`

What you must do:

1. Check pointer validity (return -1 if `q == NULL`)
2. Lock the mutex
3. Read the current valid item count in the buffer
4. Unlock the mutex and return the count

3. Understand the Snacks Scenario (PROVIDED)

File: `solution/src/bounded_buffer.c`

The snacks scenario is **PROVIDED** as a complete implementation showing how to use your bounded buffer. You should study this to understand:

- How producers (cooks) create food trays and use `bb_put ( )`
- How consumers (attendees) use `bb_take ( )` and free the trays
- How threads coordinate without explicit signaling
- Proper memory management with dynamic allocation

Example from the provided code:

```
// Producer (cook) creates a tray
food_tray_t *tray = create_food_tray(tray_id, "Pizza", cook_id);
bb_put(&snack_queue, tray); // Put pointer in buffer

// Consumer (attendee) takes and frees the tray
food_tray_t *tray = bb_take(&snack_queue); // Get pointer from buffer
LOG("Attendee took %s", tray->food_name);
free_food_tray(tray); // ⚠️ MUST FREE to avoid memory leak!
```

Part 3: Monitor-Style Readers-Writers (Phase-Fair / Batch) (30points)

Part 3 asks you to solve the Readers-Writers problem using a **Monitor style** implementation with **mutex + condition variables** (Mesa semantics), and a **phase-fair / batch** policy.

The Scenario & Requirement

Under the classic reader-priority policy of Part 1, if attendees continuously read the board, organizers may starve indefinitely, preventing urgent schedule changes from being posted. To address this, we introduce a batching policy balancing both sides:

- When **readers are waiting**, allow at most **K consecutive writers**, then **must release a batch of readers**.
- We set **K = 2** for this project.
- You must add **extra state** to track the batch/phase (e.g., consecutive writer count and current phase).

Files and Functions

File: `solution/include/sync_utils.h`

```
typedef struct {
    /* TODO: add mutex, condition variables, counters, and batch/phase state */
} rwlock_monitor_t;

int rwm_init(rwlock_monitor_t *rw);
void rwm_destroy(rwlock_monitor_t *rw);
void rwm_rlock(rwlock_monitor_t *rw);
void rwm_runlock(rwlock_monitor_t *rw);
void rwm_wlock(rwlock_monitor_t *rw);
void rwm_wunlock(rwlock_monitor_t *rw);
```

Required fields to add:

- `pthread_mutex_t m`
- `pthread_cond_t ok_to_read`
- `pthread_cond_t ok_to_write`
- `int AR, WR, AW, WW`
- `int writer_batch_count`
- `int phase`

File: `solution/src/rwlock_monitor.c`

Implement the six `rwm_*` functions. The logic should follow Mesa monitor style with a phase/batch decision in `rwm_wunlock()`.

Correctness Rules

- **No busy-waiting:** use `pthread_cond_wait`.
- **Always use `while` around waits** (spurious wakeups).
- **State changes + signaling must occur while holding the mutex.**
- **Writer priority with bounded waiting:** if writers are waiting, new readers must wait until the batch limit is reached.

Testing Your Implementation

Build the Project

```
cd solution
make clean
make
```

Run the Test Suite

After building, run the automated tests:

```
cd ../tests
./run_tests.sh -d tests
```

You can see `tests/README.md` for more details.